Basic (embeddings as geometry) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `embedding_similarity_practice.py`: "embeddings are just numbers that capture meaning" stays an abstract claim until you watch 2 similar sentences actually score closer together than unrelated ones — this is the exercise that turns it into something you measured yourself. **If not:** every later exercise in this document (chunking, retrieval, the Build Task) would be built on a formula you took on faith instead of one you watched work.

Basic Version

Approach 1 — one embedding call per sentence

```
# embedding_similarity_practice.py
import math
from openai import OpenAI

client = OpenAI()

sentences = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "Stock prices rose today.",
    "It might rain tomorrow.",
    "The bakery sells fresh bread every morning.",
]

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

embeddings = []
for sentence in sentences:
    embeddings.append(get_embedding(sentence))

for i in range(len(sentences)):
    for j in range(i + 1, len(sentences)):
        score = cosine_similarity(embeddings[i], embeddings[j])
        print(sentences[i], "|", sentences[j], "-> score:", score)
```

Expected output (scores will vary slightly by model version, but the ranking should not):

```
The cat sat on the mat. | A feline rested on the rug. -> score: 0.71...
The cat sat on the mat. | Stock prices rose today. -> score: 0.08...
The cat sat on the mat. | It might rain tomorrow. -> score: 0.12...
...
```

Sentence 0 and sentence 1 (the cat/feline pair) print the highest score by a clear margin over every other pair.

This version works correctly. It makes 5 separate API calls (one per sentence) and writes the nested loop out by hand instead of using a helper from `itertools` — both fine for a first working version.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints, `itertools.combinations`, sorted output

```
# embedding_similarity_practice.py
import math
from itertools import combinations
from openai import OpenAI

client = OpenAI()

SENTENCES: list[str] = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "Stock prices rose today.",
    "It might rain tomorrow.",
    "The bakery sells fresh bread every morning.",
]


def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding


def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)
```

```
def main() -> None:
    embeddings = {}
    for sentence in SENTENCES:
        embeddings[sentence] = get_embedding(sentence)

    scored_pairs = []
    for sentence_a, sentence_b in combinations(SENTENCES, 2):
        score = cosine_similarity(
            embeddings[sentence_a], embeddings[sentence_b]
        )
        scored_pairs.append((score, sentence_a, sentence_b))

    scored_pairs.sort(reverse=True)
    for score, sentence_a, sentence_b in scored_pairs:
        print(f"{score:.4f}  {sentence_a!r} <-> {sentence_b!r}")


if __name__ == "__main__":
    main()
```

Expected output: the same 10 pairs as Basic, but sorted, so the cat/feline pair prints first, with the highest score, instead of being buried wherever the loop happened to reach it.

Difference from Basic: full type hints on both functions. `itertools.combinations(SENTENCES, 2)` replaces the hand-written nested loop and can't accidentally compare a sentence to itself. The embeddings get stored in a dict keyed by sentence text, so each one is computed once and looked up by name instead of by matching list index. Results get sorted by score before printing, so the top pair is immediately visible instead of buried in scrollback — still one embedding call per sentence, exactly like Basic.

Approach 2 — one batched call, and the unit-vector shortcut

Story: calling `embeddings.create()` once per sentence is 5 separate network round trips for 5 short sentences — at real scale (hundreds or thousands of texts) that's a genuinely slow pipeline, one request at a time, for no reason. **If not:** the Build Task's ingest step would make one API call per document instead of one call per folder, and that difference is the gap between an ingest run that takes seconds and one that takes minutes.

```
# embedding_similarity_practice.py
import math
from itertools import combinations
from openai import OpenAI

client = OpenAI()
```

```
SENTENCES: list[str] = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "Stock prices rose today.",
    "It might rain tomorrow.",
    "The bakery sells fresh bread every morning.",
]


def get_embeddings_batch(texts: list[str]) -> list[list[float]]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=texts
    )
    embeddings = []
    for item in response.data:
        embeddings.append(item.embedding)
    return embeddings


def assert_unit_length(vector: list[float]) -> None:
    squares = 0.0
    for x in vector:
        squares = squares + x * x
    norm = math.sqrt(squares)
    assert abs(norm - 1.0) < 1e-6, f"expected a unit vector, got norm={norm}"


def fast_cosine_similarity(a: list[float], b: list[float]) -> float:
    # why: a and b are already unit-length, verified below -- skip norm()
    total = 0.0
    for x, y in zip(a, b):
        total = total + x * y
    return total


def main() -> None:
    vectors = get_embeddings_batch(SENTENCES)   # 1 API call for all 5 sentences
    assert_unit_length(vectors[0])

    sentence_to_vector = dict(zip(SENTENCES, vectors))

    scored_pairs = []
    for sentence_a, sentence_b in combinations(SENTENCES, 2):
        vector_a = sentence_to_vector[sentence_a]
        vector_b = sentence_to_vector[sentence_b]
        score = fast_cosine_similarity(vector_a, vector_b)
        scored_pairs.append((score, sentence_a, sentence_b))

    scored_pairs.sort(reverse=True)
    for score, sentence_a, sentence_b in scored_pairs:
        print(f"{score:.4f}  {sentence_a!r} <-> {sentence_b!r}")


if __name__ == "__main__":
    main()
```

Expected output: identical scores and ranking to Approach 1 — `fast_cosine_similarity` returns the same numbers as the full `cosine_similarity`, because dividing by `1.0 * 1.0` never changed anything. What changed is that this version makes exactly 1 network call total, not 5.

Approach 3 — same idea, with a text-keyed cache to avoid re-embedding duplicates

Story: a batched call already avoids 5 separate requests within one run — but a cache is what protects a program that keeps running (a chatbot, a script re-run during development) from re-embedding the exact same text twice across separate calls. **If not:** a repeated user question, or a re-run script, would silently pay for and wait on an embedding call the program already had the answer to.

```
# embedding_similarity_practice.py
import math
from itertools import combinations
from openai import OpenAI

client = OpenAI()

SENTENCES: list[str] = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "Stock prices rose today.",
    "It might rain tomorrow.",
    "The bakery sells fresh bread every morning.",
]

_embedding_cache: dict[str, list[float]] = {}


def get_embeddings_batch(texts: list[str]) -> dict[str, list[float]]:
    # how: only embed text this cache hasn't seen before -- protects
    # against a caller accidentally passing the same sentence twice
    uncached = []
    for text in texts:
        if text not in _embedding_cache:
            uncached.append(text)
    if uncached:
        response = client.embeddings.create(
            model="text-embedding-3-small", input=uncached,
        )
        for text, item in zip(uncached, response.data):
            _embedding_cache[text] = item.embedding
    result = {}
    for text in texts:
        result[text] = _embedding_cache[text]
    return result


def fast_cosine_similarity(a: list[float], b: list[float]) -> float:
    total = 0.0
    for x, y in zip(a, b):
        total = total + x * y
    return total


def main() -> None:
    sentence_to_vector = get_embeddings_batch(SENTENCES)

    scored_pairs = []
    for sentence_a, sentence_b in combinations(SENTENCES, 2):
        vector_a = sentence_to_vector[sentence_a]
        vector_b = sentence_to_vector[sentence_b]
        score = fast_cosine_similarity(vector_a, vector_b)
        scored_pairs.append((score, sentence_a, sentence_b))

    scored_pairs.sort(reverse=True)
    for score, sentence_a, sentence_b in scored_pairs:
        print(f"{score:.4f}  {sentence_a!r} <-> {sentence_b!r}")

    # how: an overlapping sentence list costs 0 extra API calls now
    get_embeddings_batch(["The cat sat on the mat.", "A brand new sentence."])


if __name__ == "__main__":
    main()
```

Expected output: same ranking as Approach 2. The proof this cache works: the second `get_embeddings_batch` call at the bottom only makes an API call for `"A brand new sentence."` — `"The cat sat on the mat."` is already in `_embedding_cache` from the first call, and is returned for free.

Difference from Approach 1, and between Approaches 2/3: Approach 1 makes 5 separate API calls and computes the full cosine similarity formula (2 `math.sqrt()` calls) for every pair. Approach 2 collapses that to 1 batched call and, having actually checked the assumption with `assert_unit_length` instead of guessing, uses `fast_cosine_similarity`'s plain dot product. Approach 3 builds on Approach 2 with a module-level cache keyed by sentence text — worth it the moment the same program might ask for the same sentence's embedding more than once (a chatbot re-embedding a repeated user question, a script re-run during development), which a plain batch call alone doesn't protect against.

Which one should you actually write? For a one-off script comparing 5 sentences, Approach 1 is completely fine — the performance difference is invisible at this size. Reach for Approach 2's batching the moment you're embedding more than a handful of texts at once; it's a real, measurable win with no added complexity. Reach for Approach 3's cache only once you know the same text might genuinely get embedded more than once in the same run — for a true production RAG pipeline, that cache usually lives in a real store (the vector database itself, from `knowledge_base_search` and the Build Task below) rather than a plain Python dict, since a dict's cache disappears the moment the process restarts.